

Rapport TP1 — Réparer une image Docker cassée

Étudiant : Nathan Ageron

Niveau : Bachelor 2

1. Diagnostic (Niveau 1)

L'analyse du Dockerfile initial a permis d'identifier 5 failles majeures empêchant son déploiement en production :

Lignes	Problème observé	Conséquence en production	Correction envisagée
Ligne 1 FROM node: 18	Image de base beaucoup trop lourde (Debian complète).	Image finale > 1 Go. Lenteur lors des pulls/déploiements et surface d'attaque très étendue (nombreuses failles potentielles - CVE).	Utiliser l'image allégée node:18-alpine.
Lignes 6-8 ENV API_KEY=...	Mots de passe et secrets écrits en clair dans l'image.	N'importe qui ayant accès à l'image peut lire les secrets via la commande docker inspect. Faille de sécurité critique.	Supprimer ces lignes du Dockerfile et passer les variables via un fichier .env lors du lancement.
Lignes 11-13 COPY . . puis npm install	Mauvaise utilisation du cache Docker. Le code est copié avant les dépendances.	La moindre modification du code source (ex: index.js) invalide le cache et force la réinstallation de tous les paquets npm, rendant les builds très lents.	Copier d'abord package.json, lancer npm install, puis copier le reste du code source.
Absente (Tout le fichier)	Aucune instruction USER n'est définie. L'application tourne en root.	En cas de compromission de l'application, l'attaquant obtient les privilèges administrateur (root) à l'intérieur du container.	Ajouter l'instruction USER node avant le CMD final.
Lignes 17-24 RUN apt-get install...	Installation d'outils de debug système inutiles (curl, wget, vim, htop...).	Alourdit inutilement le poids de l'image et met à disposition des attaquants des outils parfaits pour explorer le réseau ou télécharger des malwares.	Supprimer intégralement ce bloc d'installation.

2. Réparation et Sécurisation (Niveaux 2 & 3)

A. Fichier .dockerignore

Ce fichier empêche d'envoyer le répertoire `node_modules` local de la machine hôte vers le démon Docker, réduisant drastiquement le contexte de build et évitant les conflits d'architecture.

```
node_modules/  
.git/  
.env Dockerfile  
README.md
```

B. Dockerfile optimisé (Multi-stage build)

Ce nouveau fichier valide l'ensemble des critères de réussite : l'image pèse moins de 200MB, le cache est exploité, les secrets sont supprimés et le container tourne sous l'utilisateur `node`.

```
# Étape 1 : Build (Installation et compilation) FROM  
node:18-alpine AS builder  
WORKDIR /app  
  
# Optimisation du cache : on copie uniquement les fichiers de dépendances en premier COPY  
package*.json ./  
RUN npm install  
  
# Copie du code source et exécution du build COPY . .  
RUN npm run build  
  
# Étape 2 : Production (Image finale allégée) FROM  
node:18-alpine  
WORKDIR /app  
  
# Création du dossier data et attribution des droits à l'utilisateur non-root 'node'  
RUN mkdir -p /app/data && chown -R node:node /app  
  
# On bascule sur l'utilisateur sécurisé USER node  
  
# On récupère uniquement le résultat du build depuis l'étape précédente COPY --  
from=builder --chown=node:node /app/package*.json ./  
COPY --from=builder --chown=node:node /app/dist ./dist  
  
# On installe uniquement les dépendances de production (s'il y en a) RUN npm  
install --production
```

```
# Exposition du port et lancement de l'application EXPOSE 3000  
CMD ["node", "dist/index.js"]
```

3. Preuves de vérification

Les commandes de validation démontrent la résolution des problématiques :

- `docker build -t tp1:corrige .` : Le build réussit grâce au cache (npm install est *CACHED* lors de modifications de code).
- `docker images` : L'image `tp1:corrige` pèse environ 175 Mo (inférieur à la limite des 200 Mo).
- `docker run --rm tp1:corrige whoami` : Retourne `node`, prouvant la sécurisation des privilèges.
- `docker inspect tp1:corrige` : Les clés API et mots de passe ne sont plus exposés dans la section `Env`.